

前言：

目前的語音辨認，大家最常用的 Feature Extraction 方法就是 MFCC(Mel Frequency Cepstrum Coefficient)。

下圖是簡略的 MFCC Flow Diagram。

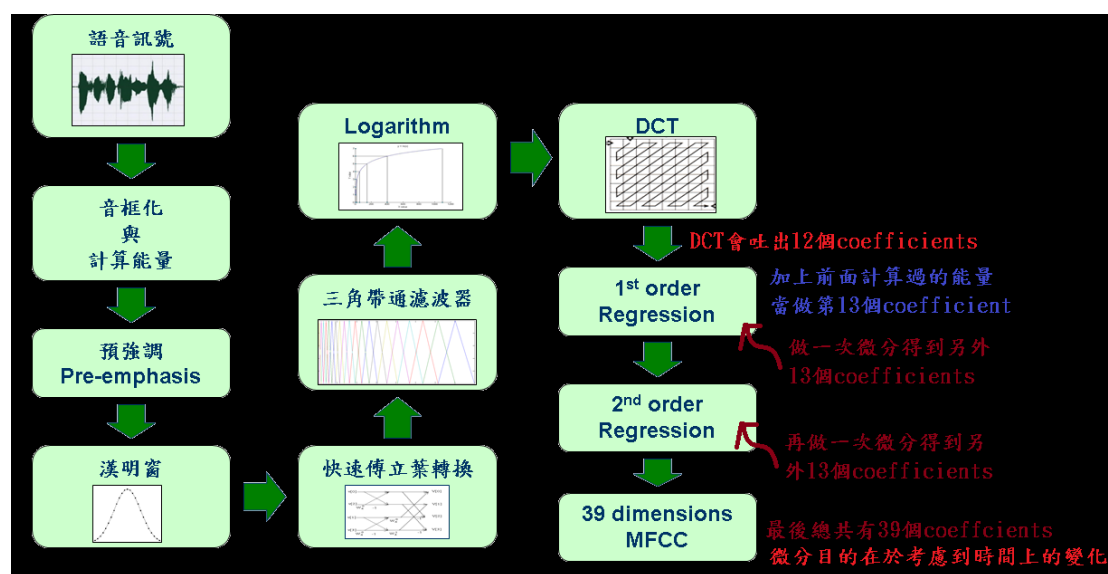


Fig 1: 修改自張智星老師實驗室學生之投影片。

而我的題目選擇將「DCT(Discrete Cosine Transform)的部分實作出來」。以下有三個假設：

- (1) 在此假設 DCT 輸入 24 個值，並輸出 12 個值。
- (2) 假設 filter banks 有 24 個，因此經過 log 運算後丟給 DCT 的值有 24 個。
- (3) 由於語音辨認只需要 12 個 DCT 的值(低頻部分)，剩下的 24 個值(高頻部分)可以不用計算。

實驗題目：

1D- DCT (Discrete Cosine Transform)

實驗環境：

使用課程所提供的模擬器(Emulator)。

記憶體:2G

核心:Dual Core

資料夾內容介紹：

名稱	說明
 DCT_CPU.cu	host 端的程式，DCT 在 CPU 上執行
 DCT_GPU_way1.cu	device 端的程式，DCT 在 GPU 上執行。 使用第一種資料切割(Thread 數量大於 Task 數量)
 DCT_GPU_way2.cu	device 端的程式，DCT 在 GPU 上執行。 使用第一種資料切割(Thread 數量小於 Task 數量)
cepstrumCPU.txt	host 端、device 端 way1 和 device 端 way2 的程式分別丟出的結果(此 DCT 丟出的值有個特別的名稱叫做 Cepstrum，中文叫倒頻譜)
cepstrumGPUway1.txt	
cepstrumGPUway2.txt	

DCT 定義:

To make DCT a orthonormal transform, we define a coefficient

$$a[n] = \begin{cases} \sqrt{1/N} & n = 0 \\ \sqrt{2/N} & n = 1, 2, \dots, N-1 \end{cases}$$

so that the DCT now becomes

$$X[n] = a[n] \sum_{m=0}^{N-1} x[m] \cos\left(\frac{(2m+1)n\pi}{2N}\right) = \sum_{m=0}^{N-1} x[m] c[n, m] \quad (n = 0, \dots, N-1)$$

其中，N 是 input 點數，m 是第幾筆資料，n 是底幾個 coefficient。

驗證 DCT 結果:

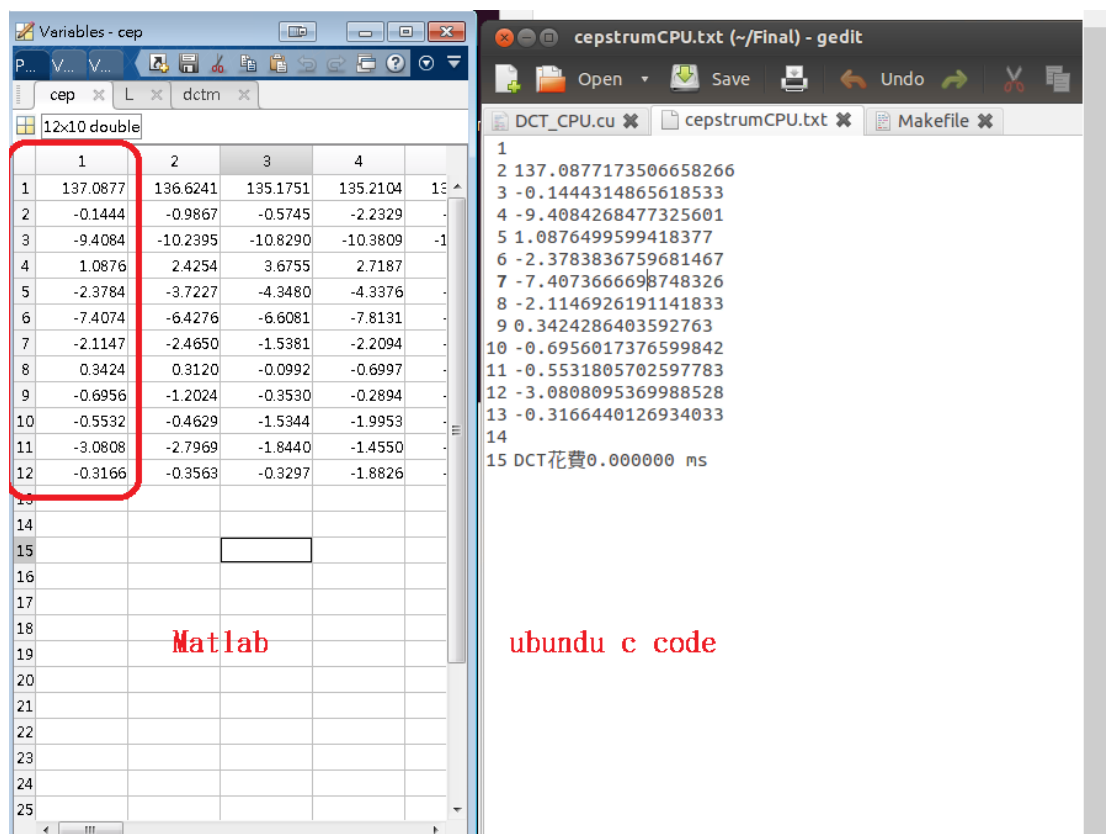


Fig 2: host 端算出來的結果與 Matlab 一致。Matlab 不只一組 cepstrum，是因為它一次計算多組音框的值。為了方便，我先計算一個音框的值。

host 端程式說明：

```
void dct(double *DCTMatrix, double *Matrix, int N, int NumCeps){  
    int n, m;  
    double a0 = sqrt(1./N);  
    double a = sqrt(2./N);  
    for (n = 0; n < NumCeps; n++) {  
        for (m = 0; m < N; m++) {  
            DCTMatrix[n] += Matrix[m] * cos(M_PI/((float)N)*(m+1./2.)*n);  
        }  
        if(n == 0)  
            DCTMatrix[n] = DCTMatrix[n]*a0; 讓第一個coefficient乘上  $\sqrt{\frac{1}{N}}$   
        else  
            DCTMatrix[n] = DCTMatrix[n]*a; 讓其餘coefficients乘上  $\sqrt{\frac{2}{N}}$   
    }  
}
```

以第一個 coefficient 為例，

需要 24 個值相累加，最後才乘上一個常數。如下圖所示：

```
DCTMatrix[0] = Matrix[0] * cos(const.*(0 + 0.5)*0);  
              + Matrix[1] * cos(const.*(1 + 0.5)*0);  
              + Matrix[2] * cos(const.*(2 + 0.5)*0);  
              -  
              -  
              -  
              + Matrix[22] * cos(const.*(22 + 0.5)*0);  
              + Matrix[23] * cos(const.*(23 + 0.5)*0);  
  
DCTMatrix[0] = DCTMatrix[0] * sqrt(1./N)
```

以 host 端程式寫法，必須算完第一個 coefficient 的值，再算第 2 個
值到算完 12 個 coefficients 為止。

資料分割方式：

way1: 平行計算 coefficients 的值。(Threads 數量大於 Task 數量)

宣告 <<<1,32>>> 總共一個 block，每個 block 有 32 個 thread。

步驟一：

第一次先開 24 個 threads 將所有 coefficients 個別需要累加的 24 個值算出結果並存起來。因為 Threads(32)數量比 task(24)大，僅使用 24 個 thread 對個別的 element of array 做計算。

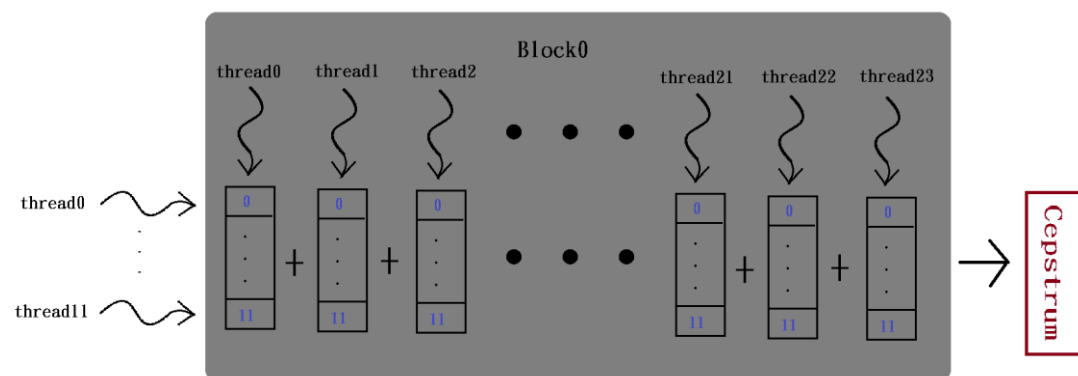
步驟二：

開 12 個 threads 將存起來的值個別加起來得到不同的 12 個累加值。

步驟三：

將 coefficients 乘上它門對應的常數值。

如下圖所示：



CUDA code:

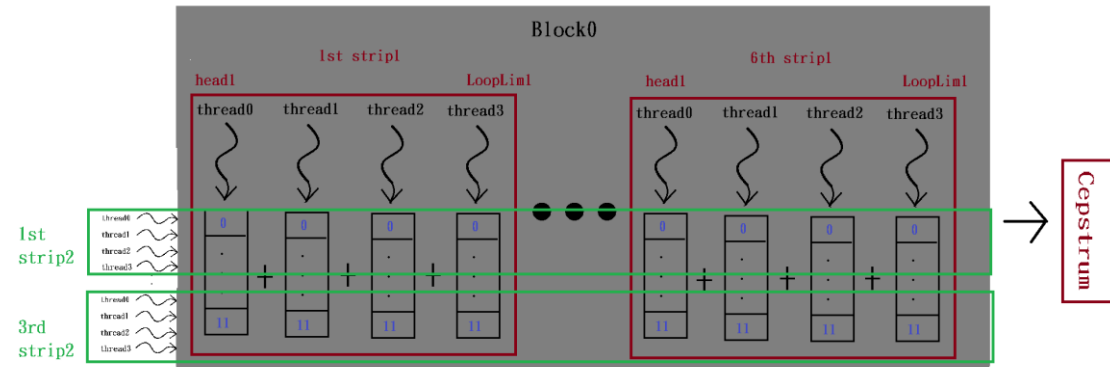
```
__global__ void dct_gpu(float *DCTMatrix, float *Matrix, int N, int NumCeps)
{
    int n, m;
    float a0 = sqrt(1./N);
    float a = sqrt(2./N);
    float MatTemp[12][24];
    int idx = blockIdx.x*blockDim.x+threadIdx.x;

    if(idx < N){//N=24
        for (n = 0; n < NumCeps; n++) {
            MatTemp[n][idx] = Matrix[idx] * cos(M_PI/((float)N)*(idx+1./2.)*n);
        }
    }
    if(idx < NumCeps){
        for (m = 0; m < N; m++) {
            DCTMatrix[idx] += MatTemp[idx][m];
        }
        if(idx == 0)
            DCTMatrix[idx] = DCTMatrix[idx]*a0;
        else
            DCTMatrix[idx] = DCTMatrix[idx]*a;
    }
}
```

way2: 平行計算 coefficients 的值。(Threads 數量小於 Task 數量)

宣告 <<<1,4>>> 總共一個 block，每個 block 有 4 個 thread。

步驟與 way1 一樣，差在同樣的 thread 分批對所有的 element 做運算。



CUDA code:

```
__global__ void dct_gpu(float *DCTMatrix, float *Matrix, int N, int NumCeps)
{
    int n, m;
    float a0 = sqrt(1./N);
    float a = sqrt(2./N);
    float MatTemp[12][24];
    //Task=24
    int i=0;
    int TotalThread = blockDim.x*gridDim.x;
    int stripe1 = N / TotalThread;
    int head1 = (blockIdx.x*blockDim.x + threadIdx.x)*stripe1;
    int LoopLim1 = head1+stripe1;
    //Task=12
    int stripe2 = NumCeps / TotalThread;
    int head2 = (blockIdx.x*blockDim.x + threadIdx.x)*stripe2;
    int LoopLim2 = head2+stripe2;

    for(i=head1 ; i<LoopLim1 ; i++)
    {
        for (n = 0; n < NumCeps; n++) {
            MatTemp[n][i] = Matrix[i] * cos(M_PI/((float)N)*(i+1./2.)*n);
        }
    }
    for(i=head2 ; i<LoopLim2 ; i++)
    {
        for (m = 0; m < N; m++) {
            DCTMatrix[i] += MatTemp[i][m];
        }
        if(i == 0)
            DCTMatrix[i] = DCTMatrix[i]*a0;
        else
            DCTMatrix[i] = DCTMatrix[i]*a;
    }
}
```

Makefile:

```
1 CC=g++
2 LINKER_DIRS=-L/usr/local/cuda/lib
3 LINKER_FLAGS=-lcudart -lcuda
4 NVCC=nvcc
5 CUDA_ARCHITECTURE=20
6 OCELOT='OcelotConfig -l'
7
8 all:
9     $(NVCC) -c DCT_GPU_way1.cu -arch=sm_$(CUDA_ARCHITECTURE) -I .
10    $(NVCC) -c DCT_GPU_way2.cu -arch=sm_$(CUDA_ARCHITECTURE) -I .
11    $(NVCC) DCT_CPU.cu -c -I .
12
13    $(CC) DCT_GPU_way1.o -o dctGPUway1 $(LINKER_DIRS) $(OCELOT)
14    $(CC) DCT_GPU_way2.o -o dctGPUway2 $(LINKER_DIRS) $(OCELOT)
15    $(CC) DCT_CPU.o -o dctCPU $(LINKER_DIRS) $(OCELOT)
16
17 clean:
18    rm -f DCT_CPU.o DCT_GPU_way1.o DCT_GPU_way2.o dctCPU dctGPUway1 dctGPUway2 kernel-times.json
19    rm cepstrumCPU.txt cepstrumGPUway1.txt cepstrumGPUway2.txt
```

框選為執行檔名稱。dctCPU, dctGPUway1 和 dctGPUway2。